COMM.SYS.400-2022-2023-1
Matlab Project Work

Elementary Baseband Equivalent Digital Communication System

1 General Information

The **final deadline** for returning the complete project work is **25th of May, 2023**. The credit points (5 ECTS credits) will be transferred only when both the exam (or mid-term exams) and the project are completed and approved. The work should be returned to the advisor by email (see the supervisor instructions below!). The email should include the following items:

- name, student number, and email address,
- Matlab code for the simulation(s) as m-file as attachment (The m-file should contain your name, student number and email on the first line; the file name for the returned work should be your last name, i.e. `your_last_name.m`.) and
- answers to the given questions (a couple of short sentences per question).

Good programming discipline should be followed when writing the Matlab-code. This means that the variable names should be logical, the code must be commented and it should be written in such a way that it is easy to follow and understand. Figures should have appropriate titles and axis labels and use of `axis` command is recommended (see help: `title`, `xlabel`, `ylabel`, `axis`). If you have any questions concerning the work or practical arrangements, please contact the project advisor:

Roman Klus

room: TD 321
email: roman.klus@tuni.fi

2 Task

In this Matlab-project, you should simulate a M-quadrature amplitude modulation (M-QAM) or M-phase-shift keying (M-PSK) digital communication system (or actually a baseband equivalent system since complex symbol alphabets can only be used in bandpass transmission in practice) including transmitter, channel, and receiver functions. The goal in this project work is to learn and demonstrate the meaning of some essential blocks in digital communication systems. After you have completed the work, you should know why blocks like error control coding, transmit filter, receive filter, equalizer, etc. are used and how to simulate them by using Matlab. Figure 1 shows the block diagram of the simulation system.

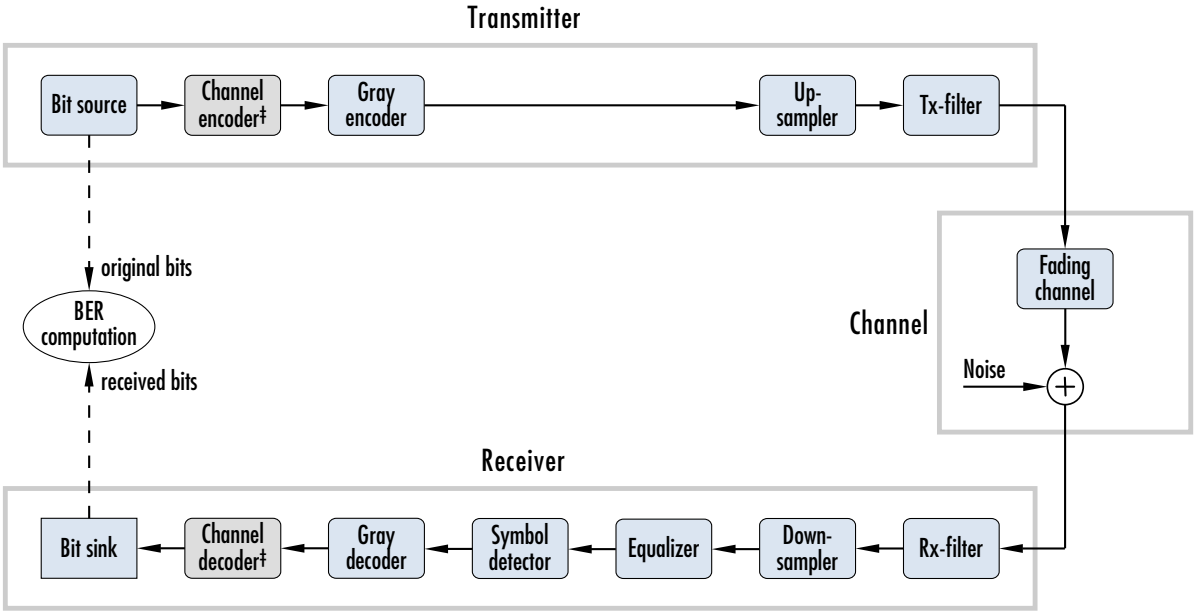

Figure 1: Block diagram of digital communication system to be simulated. The error control encoding and decoding part are to be skipped first and are added after the overall system is running. See Table 1 for the system parameters.

2.1 Transmitter

The symbol rate of digital communication system is specified to $R_s = 40\,\text{MHz}$ ($R_s = 1/T = 40 \times 10^6\,\text{symbols/s}$). Further parameters, such as the particular M-QAM or M-PSK modulation, the over-sampling factor and the sampling frequency F_s, for pulse-shaping filtering, are given in Table 1.

Note: The parameters of the system differ corresponding to the last digit of your student number. Table 1 can be found on page 6.

1. Generate $\log_2(M) \times 100\,000$ random bits.
2. Here comes the channel encoder.
3. Use the Gray mapping to generate 100 000 M-QAM or M-PSK symbols. Matlab's `bin2gray` is handy for this (see `help bin2gray`); it may help to generate the 100 000 symbol indices that can be plugged into the M-QAM/M-PSK symbol alphabet to generate the symbols. When using `bin2gray`, take into account that the output of `bin2gray` ranges from 0 to $M-1$ and not from 1 to M.

⊟ Plot the constellation diagram of the generated symbols and verify their correctness.

4. Up-sample the symbols using the corresponding over-sampling factor $F_\text{s}T$ from Table 1.
5. Use a square square root raised cosine (RRC) filter to carry out pulse-shape filtering for the symbols. The impulse response of the filter can be generated by Matlab's `rcosdesign`-function, see `help rcosdesign`. Its parameters, the roll-off factor, α, the duration of the filter in symbols, N_d, and the over-sampling factor, $F_\text{s}T$, are given in Table 1.

- Do not forget to correct for the delay of the filter in order to keep the pulse-shaped sequence time-synchronized with the original symbols (this will help you later, in error probability evaluations), see Matlab-exercise 1 for an example how this can be achieved.

⊟ Plot the eye-diagram of the generated signal.

- The length of the signal blocks should be nF_sT, where n is an integer (e.g., $n = 2$ or $n = 3$).
- Here it is enough to plot the eye-diagram using only the real part of the signal. Why?

⊟ Examine the spectral content of the transmitted signal by plotting (i) the amplitude response of the transmit filter and (ii) the amplitude spectrum of the actual transmitted signal. Compare these two.

- When plotting the amplitude response and spectrum, include both negative and positive frequencies (i.e., $-F_s/2, \ldots, F_s/2$) and scale the frequency axis correctly.

Question 1 *What is the effect of the roll-off factor on the transmitted signal in general?*

Question 2 *Why the eye-diagram of the transmit signal does not show pure/perfect eye opening?*

2.2 Transmission over Channel

1. Use the corresponding complex channel impulse response, provided in Table 1, and pass the transmitted signal through the channel.

   - Do not forget to correct for the delay of the channel in order to keep the channel output signal time-synchronized with the original symbol sequence. Regard therefore the channel's impulse response as a finite-impulse response (FIR) filter, find the strongest tap of the filter to find the delay of the channel and correct for it.

2. Add white Gaussian noise to the resulting signal, see the block-diagram in Figure 1. Adjust the noise power to obtain an in-band signal-to-noise ratio (SNR) of 15 dB.

⊟ Plot the frequency response of the channel – including both the amplitude and the phase response.

⊟ Plot the amplitude spectrum of the received signal after channel filtering and adding the noise. Compare it with the amplitude spectrum of the transmitted signal.

Question 3 *Is it feasible to use a fixed impulse response as channel model for a radio channel in mobile communication systems? Why?/Why not?*

2.3 Receiver

1. Filter the received signal with a filter matched to the used Tx filter (square-root raised-cosine).

   - Do not forget to correct for the delay of the filter in order to keep the filtered sequence time-synchronized with the original Tx symbols.

⊟ Plot the eye-diagram of the received signal and compare it to the eye-diagram of the transmitted signal. Repeat the same such that you momentarily exclude the channel model and additive noise. Explain what you observe.

2. Down-sample the signal to the symbol rate R_s.

⊟ Plot the symbol constellation of the downsampled signal (i.e., the complex samples after receiver filtering and down-sampling), with channel model and noise again included in your simulation chain.

3. Use the least mean squares (LMS) algorithm-based equalizer for the channel equalization. Intuitively, you try to find the inverse response (roughly) of the effective channel by using a known training sequence. The number of iterations and the order/length of the equalizer depend on the situation, good values to start with are, e.g. 1000 to 10 000 iterations, step-size $\mu = 0.01$ and the order/length between 10 to 30. First, try to get the equalizer to work *without noise*. Then, include the noise and tune the algorithm if necessary.

⊟ Plot the LMS error and check if the LMS algorithm is converging. If you have problems with the convergence, try changing the iteration step-size, number of iterations, and/or the equalizer order/length.

   When the equalizer filter is found, equalize the whole received sample sequence by filtering it with the equalizer filter (with the impulse response of the equalizer).

   - Do not forget to correct for the delay of the equalizer, when doing the final equalization filtering with the obtained impulse response.

⊟ Plot the amplitude and phase response of the obtained equalizer filter into the plot of the frequency response of the channel. Compute the total frequency and phase response of the channel and the equalizer and plot it in the same figure.

⊟ After having a properly working equalizer, plot the constellation of the equalized signal on top of the constellation of the unequalized received signal. Plot also the originally generated symbols as reference on top.

4. Detect the received equalized samples using a symbol-by-symbol minimum distance detector.

5. Use the Gray decoder to obtain the coded bits. In case you use Matlab's `gray2bin` function, consider that it expects the principle input parameter ranging from 0 to $M-1$, and not from 1 to M.

6. Here comes the channel decoder.

7. Then compute the bit error rate (BER) by comparing the obtained bits with the original transmitted bits. You have to be careful that both bit sequences are time-synchronized; that is, the original and the received sequences (vectors) contain the actually corresponding bits at the same positions.

   - In principle, if you have taken properly into account the delay of each individual filtering stage in the simulation chain, then things should go correctly automatically – but be careful. If having problems with time synchronization, use momentarily a noiseless system (set SNR to be e.g. 100 dB) to be able to better debug your simulator.

Question 4 *What is the meaning of the eye-diagram? What can be observed in an eye-diagram?*

Question 5 *What is the idea of matched filtering in general? Consider true matched filter (matched to the received pulse) versus matched filter to the transmitted pulse?*

Question 6 *What is the idea of an equalizer in a digital communication system?*

2.4 Error control coding

Now, error control is incorporated into the existing system and the generated bits are encoded. Therefore, the channel encoder part will be implemented in the transmitter (see sect. 2.1 item 2) and the corresponding decoder will be implemented in the receiver (see sect. 2.3 item 6).

1. Encode the original bit sequence with a Hamming code that maps $k = 4$ source bit to $n = 7$ coded bit – H(7,4), with help of the generator matrix $\mathbf{G} = (\mathbf{I}_k|\mathbf{P})$. The parity array of the generator matrix reads

$$\mathbf{P} = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix},$$

   and $\mathbf{I}_k$ denotes the identity matrix of dimension k.

2. Decode the received, coded bit sequence and try to correct errors using a decoding table containing the n single bit error words and the corresponding single bit error syndromes. This set of syndromes can be generated with help of the parity check matrix $\mathbf{H} = (\mathbf{P}^T|\mathbf{I}_{n-k})$.

   Ignore/discard the bits at the end of the received sequence that are not multiples of the codeword length.

3. Extend your program to repeat the simulation for SNRs from 0 dB to 20 dB.

4. Run the whole simulation with and without error control coding and compute the BERs for SNR from 0 dB to 20 dB.

⊟ Plot the obtained BER values obtained from the system with coding and without coding as a function of the SNR, read the obtained gain at BER of 1×10^{-3}. Explain what you observe. How this coding gain could be achieved? What is the trade-off here?

Question 7 *What performance would you expect over the given SNR range using a soft-decision decoder instead of the hard-decision decoder. Why, what is the trade-off?*

Table 1: Simulation parameters which depend on the last digit of your student number. Symbol rate $R_s = 1/T = 40\,\text{MHz}$ is a common parameter for everyone.

Last digit of student number	Modulation (M)	Oversampling factor ($F_s T$)	RRC-filter roll-off factor (α)	Duration of RRC-filter in symbols (N_d)	Impulse response of channel model
0	16-QAM	3	0.20	16	Model 1
1	4-QAM	3	0.25	20	Model 2
2	16-QAM	3	0.40	16	Model 3
3	8-PSK	2	0.10	20	Model 4
4	16-QAM	4	0.30	10	Model 5
5	16-QAM	2	0.20	10	Model 6
6	4-QAM	4	0.40	20	Model 7
7	4-PSK	2	0.30	16	Model 8
8	16-QAM	2	0.10	10	Model 9
9	8-PSK	4	0.20	16	Model 10

Table 2: Different channel models given by the coefficients of their impulse response.

Model 1:	`h = [ 0.2178 + 0.8127i,  0.2166 + 0.7454i, -0.2924 - 0.5189i,  0.2179 - 0.3899i, -0.5940 + 0.5274i]`
Model 2:	`h = [-0.4863 - 0.4696i, -0.8091 + 0.3738i,  0.3908 + 0.3593i, -0.1731 + 0.4118i,  0.0956 - 0.3014i]`
Model 3:	`h = [ 0.5399 - 0.4069i, -0.8091 + 0.3738i,  0.3908 + 0.3593i, -0.1731 + 0.4118i,  0.0956 - 0.3014i]`
Model 4:	`h = [-0.4781 - 0.4781i, -0.3339 + 0.8264i,  0.0278 - 0.5302i,  0.2233 - 0.3868i,  0.0221 + 0.3155i]`
Model 5:	`h = [ 0.8230 + 0.1749i, -0.4495 + 0.4221i, -0.7186 - 0.2142i,  0.7079 - 0.0099i, -0.0913 - 0.7891i]`
Model 6:	`h = [ 0.5718 - 0.2667i, -0.4363 - 0.6420i,  0.5645 + 0.1900i,  0.0529 + 0.4435i, -0.1266 - 0.2898i]`
Model 7:	`h = [-0.5261 - 0.4737i, -0.6539 + 0.6056i,  0.5204 - 0.1049i,  0.3381 + 0.2919i,  0.1877 + 0.2545i]`
Model 8:	`h = [ 0.5180 + 0.6630i, -0.3951 + 0.6681i,  0.5363 - 0.2593i, -0.2651 + 0.3595i,  0.7090 + 0.3581i]`
Model 9:	`h = [-0.3878 - 0.5538i, -0.4723 + 0.7558i,  0.0278 - 0.5302i,  0.2233 - 0.3868i,  0.0221 + 0.3155i]`
Model 10:	`h = [ 0.8077 + 0.3767i, -0.4628 + 0.5250i,  0.5862 + 0.1055i, -0.4393 + 0.0806i, -0.2813 - 0.1445i]`